

An Approximate String Matching Algorithm Based upon the Candidate Elimination Method

C. W. Lu and R. C. T. Lee

Department of Computer Science and Information Engineering
National Chi Nan University, Nantou, Taiwan, ROC
{s95321512, rctlee}@ncnu.edu.tw

Abstract

In this paper, we consider the approximate string matching problem. We give a method to eliminate candidate locations in text T as there can be no substring S starting from those locations such that the edit distance between S and pattern P is smaller than or equal to a specified error bound k . Our method is simple to implement. Experimental results show that our method is effective, especially in the case when we perform natural language searching. For instance, for a DNA type data, when the text is 10000 characters long, patterns are 40 characters long and $k=3$, in average, 73% of locations are eliminated by our method. As for a English text with 4500 characters, for English patterns with 20 characters, in average, 99.9% of locations are eliminated if $k=1$ and 73.3% for $k=6$.

1 Introduction

In this paper, we are interested in the approximate string matching problem defined as follows: Given a text $T = t_1 t_2 \dots t_n$, a pattern $P = p_1 p_2 \dots p_m$ and a error bound k , find all the substrings of T whose edit distances with P are less than or equal to k . The edit distance between A and B , denoted $ED(A, B)$, is the minimum number of insertion, deletion or substitution operations needed to transform B into A . Edit distance can be computed by using dynamic programming method. Much research focuses on algorithms to find the edit distance between two strings, and many of them [1, 3, 4, 5, 7, 8, 9, 10, 11, 12, and 13] use the following rule: Consider the two strings $A=A_1 A_2$ and $B=B_1 B_2$, if $A_2=B_2$ and $ED(A_1, B_1) \leq k$, then $ED(A, B) \leq k$.

There are two approaches to solve our problem. The first approach, called Approach 1 by us, is a local approach based upon the dynamic programming method to compute edit distance [10]. For every possible location i , we test whether there exists a substring S of

T starting from this location such that $ED(S, P) \leq k$. Another approach [13], named Approach 2, is a global approach which can solve our problem in $O(nk \lceil m/w \rceil)$ time-complexity where w is the word size.

So far as worst case time-complexity is concerned, Approach 2 is better than Approach 1. But, it is possible that for a certain location i , there is no substring S starting from location i such that $ED(S, P) \leq k$. Recent research shows that the problem can be efficiently solved by Approach 1 algorithms if many such candidate locations can be eliminated [8, 9, and 12]. Therefore, it is meaningful to investigate the candidate elimination schemes because it will make Approach 1 algorithms very efficient in average case time-complexity. In this paper, we shall give a method to find such candidate locations.

2 Our Approach

The following lemma is relevant to our approach and has been used in many papers [2, 6, 8, 9, and 12].

Lemma 1

Given two strings A and B , and a error bound k . If $ED(A, B) \leq k$, then $|B| - k \leq |A| \leq |B| + k$.

By Lemma 1, we can prune some substrings of T with size smaller than $m - k$ or larger than $m + k$ out of our consideration. Let S_x^i be the substring $t_i t_{i+1} \dots t_{i+x-1}$ of T with size x . For every position i of T , we only consider the substrings $S_{m-k}^i, S_{m-k+1}^i, \dots, S_{m+k}^i$. We now focus on the substrings S_{m-k}^i and S_{m+k}^i .

Let $N_c(S)$ denote the number of character c 's appearing in S , Σ denote the set of alphabets in P and T , C_1 be the set of all alphabets such that $N_c(S_{m-k}^i) > N_c(P)$ and $c \in \Sigma$, and C_2 be the set of all alphabets such that $N_c(P) > N_c(S_{m+k}^i)$ and $c \in \Sigma$.

Our approach is based upon the following lemmas:

Lemma 2 If $\sum_{c \in C_1} (N_c(S_{m-k}^i) - N_c(P)) > k$, then $ED(S_x^i, P) > k$ for $m-k \leq x \leq m+k$.

Proof. Since $c \in C_1$, by definition, $N_c(S_{m-k}^i) > N_c(P)$. Consider the substrings $S_{m-k+1}^i, S_{m-k+2}^i, \dots, S_{m+k}^i$. We have $N_c(S_x^i) > N_c(S_{m-k}^i) > N_c(P)$ if $m-k \leq x \leq m+k$ because S_{m-k}^i is a prefix of the substring S_x^i . By assumption, $\sum_{c \in C_1} (N_c(S_{m-k}^i) - N_c(P)) > k$. Then $\sum_{c \in C_1} (N_c(S_x^i) - N_c(P)) > \sum_{c \in C_1} (N_c(S_{m-k}^i) - N_c(P)) > k$, for $m-k \leq x \leq m+k$. Thus, $\sum_{c \in C_1} (N_c(S_x^i) - N_c(P)) > k$. That is, we need at least $k+1$ operations of insertion on P to transform P into S_x^i , for $m-k \leq x \leq m+k$. Thus $ED(S_x^i, P) > k$, for $m-k \leq x \leq m+k$.

The following lemma can be proved by using similar reasoning.

Lemma 3 If $\sum_{c \in C_2} (N_c(P) - N_c(S_{m+k}^i)) > k$, then $ED(S_x^i, P) > k$ for $m-k \leq x \leq m+k$.

Theorem 1 If $\sum_{c \in C_1} (N_c(S_{m-k}^i) - N_c(P)) > k$ or $\sum_{c \in C_2} (N_c(P) - N_c(S_{m+k}^i)) > k$, we can eliminate position i of T . That is, we prune all substrings S_x^i , for $x > 0$ out of consideration.

Example:

Let $P = "ecfdccabfa"$, $T = "egfaaaebfaacgdeg g g ecgagbgabeffadfbfgddeg"$, $k=2$ and $\Sigma = \{a, b, c, d, e, f, g\}$. We first compute the values $N_a(P) = 2$, $N_b(P) = 1$, $N_c(P) = 3$, $N_d(P) = 1$, $N_e(P) = 1$, $N_f(P) = 2$, $N_g(P) = 0$. We can eliminate all positions of

T except Position 3 because only Position 3 of T does not satisfy the conditions in Theorem 1. For example, consider $i=5$. $S_{m+k}^5 = S_{12}^5 = "aaebfaacgdeg"$. We have $N_a(S_{m+k}^5) = 4$, $N_b(S_{m+k}^5) = 1$, $N_c(S_{m+k}^5) = 1$, $N_d(S_{m+k}^5) = 1$, $N_e(S_{m+k}^5) = 2$, $N_f(S_{m+k}^5) = 1$, and $N_g(S_{m+k}^5) = 2$. Thus, we have $N_c(P) > N_c(S_{m+k}^5)$ and $N_f(P) > N_f(S_{m+k}^5)$. $\sum_{x=c \text{ or } f} (N_x(P) - N_x(S_{m+k}^5)) = (N_c(P) - N_c(S_{m+k}^5)) + (N_f(P) - N_f(S_{m+k}^5)) = (3-1) + (2-1) = 3 > k$.

Therefore, Position 5 can be eliminated.

Based on Theorem 1, we present an algorithm to solve the approximate string matching problem.

I. Preprocessing Phase

In the preprocessing phase, we use an array $NumP(c)$ to record the number of character c in P , $N_c(P)$, $\forall c \in \Sigma$. This can be done by a linear scan of P .

II. Searching Phase

For a position i of T , we first examine whether the position i satisfies any one of the conditions in Theorem 1. If it does, we would not check all the substrings S_x^i , $x > 0$; otherwise, we check the substrings S_x^i , $m-k \leq x \leq m+k$, by the dynamic programming method. Note that we only have to compute the edit distance between $T(i, i+m+k-1)$ and $P(1, m)$.

Algorithm A:

Input: $P(1, m)$, $T(1, n)$, error bound k and $NumP(c)$.

Output: All substrings of T whose edit distances with P are smaller than or equal to k .

```

1  Set  $NumT_1(c) = 0$  and  $NumT_2(c) = 0, \forall c \in \Sigma$ .
2  Set  $i = 1$ .
3  While( $i \leq m+k$ ) do
4      If  $i \leq m-k$ , set  $NumT_1(t_i) = NumT_1(t_i) + 1$ .
5      If  $i \leq m+k$ , set  $NumT_2(t_i) = NumT_2(t_i) + 1$ .
6      Set  $i = i + 1$ .
7  End While
8   $Diff_1(c) = NumT_1(c) - NumP(c), \forall c \in \Sigma$ .
9   $Diff_2(c) = NumP(c) - NumT_2(c), \forall c \in \Sigma$ .
10 Set  $i = 1$ .
11 While( $i \leq n$ ) do
12     Set  $Sum_1 = 0$  and  $Sum_2 = 0$ .
13      $\forall c \in \Sigma$ , if  $Diff_1(c) > 0$ ,  $Sum_1 = Sum_1 + Diff_1(c)$ , and if  $Diff_2(c) > 0$ ,  $Sum_2 = Sum_2 + Diff_2(c)$ .
14     If  $Sum_1$  and  $Sum_2$  are not larger than  $k$ , compute the edit distance between the substring  $T(i, i+m+k-1)$  and  $P(1, m)$  by Dynamic Programming Method. Output the substring  $S_x^i$  if  $ED(S_x^i, P) \leq k$ .
15     Set  $Diff_1(t_i) = Diff_1(t_i) - 1$ , and  $Diff_2(t_i) = Diff_2(t_i) + 1$ .
16     Set  $Diff_1(t_{i+m-k}) = Diff_1(t_{i+m-k}) + 1$ , and  $Diff_2(t_{i+m-k}) = Diff_2(t_{i+m-k}) - 1$ .
17     Set  $i = i + 1$ .
18 End While

```

3 Experiments

In our experiments, we examine the percentage of positions of T which we can eliminate by using our method.

Experiment 1: DNA Type Data

In the first experiment, we generate P with size m and T with size n randomly by using an alphabet Σ with $|\Sigma|=4$. We set $n=10000$, $m = \{20, 30, 40\}$, and $1 \leq k \leq 3$. Each result is the average of 1000 experiments with different P 's and one T . The result is shown in Table 3-1.

Table 3-1 (Percentage of positions eliminated)

	$m=20$	$m=30$	$m=40$
$k=1$	93.8%	96.3%	97.6%
$k=2$	74.5%	83.9%	89.0%
$k=3$	46.7%	63.1%	73.1%

Our approach is very efficient if the alphabet size is large. Consider the situation when we are given a rather lengthy English essay and a sentence which we think it exists in this essay, but not sure. Thus we want to search the essay and we insist that the search sentence should not be too different from our sentence. We may set an error k . For instance, the essay is "When I am dead, my dearest, sing no songs for me; plant no roses at my head, nor shady cypress tree. Be the green grass above me with showers and drew drops wet. And if thou will, remember, and if thou will not, forget.", and our sentence is "nor shady cherry tree". If we set $k=5$, we will be able to find the sentence "nor shady cypress tree". But, during the search, out of 218 locations, only 41 locations need to be considered. In the second experiment, we tested our algorithm on English essays.

Experiment 2 Natural Language Data

In the second experiment, we chose an English article with 4500 characters, and we randomly selected sentences with 20 characters from the article. 1000 sentences were chosen. We set $1 \leq k \leq 6$. The result of the percentage of locations eliminated for each k is shown in Table 3-2. As can be seen, our approach is quite good for such kind of data.

Table 3-2 (Percentage of positions eliminated)

$k=1$	99.9%
$k=2$	99.8%
$k=3$	99.5%
$k=4$	97.8%
$k=5$	90.7%
$k=6$	73.3%

4 Concluding Remarks and Future Research

We have proposed a method to eliminate candidates in a text string for approximate string matching. Experimental results show that our method is feasible and very effective especially when we perform natural language processing. We plan to work on the average case time complexity analysis of our algorithm in the future. We also plan to extend our idea to

tackle the exact string matching problem. We have already done some experiments and it seems that our idea is very good for natural language data.

References

- [1] Amir, A., Keselman, D., Landau, G. M., Lewenstein, M., Lewenstein, N. and Rodeh, M., Text Indexing and Dictionary Matching with One Error, *Journal of Algorithms*, Vol. 37, 2000, pp.309-325.
- [2] Fredriksson, K., Navarro, G., Average-Optimal Multiple Approximate String Matching, *ACM Journal of Experimental Algorithmics*, Vol. 9, No. 1.4, 2004, pp.1-47.
- [3] Hyyrö, H., Bit-parallel approximate string matching algorithms with transposition, *Journal of Discrete Algorithms*, Vol. 3, 2005, pp.215-229.
- [4] Huynh, T. N. D., Hon, W. K., Lam, T. W. and Sung, W. K., Approximate String Matching Using Compressed Suffix Arrays, *Theoretical Computer Science*, Vol. 352, 2006, pp.240-249.
- [5] Holub, J., Melichar, B., Approximate string matching using factor automata, *Theoretical Computer Science*, Vol. 249, 2000, pp.305-311.
- [6] Hyyrö, H. and Navarro, G., Bit-parallel Witnesses and their Applications to Approximate String Matching, *Algorithmica*, Vol. 41, No. 3, 2005, pp.203-231.
- [7] Landau, G. and Vishkin, U., Fast Parallel and Serial Approximate String Matching, *Journal of Algorithms*, Vol. 10, 1989, pp.157-169.
- [8] Navarro, G. and Baeza-Yates, R., Very fast and simple approximate string matching, *Information Processing Letters*, Vol. 72, 1999, pp.65-70.
- [9] Navarro, G. and Baeza-Yates, R., A Hybrid Indexing Method for Approximate String Matching, *Journal of Discrete Algorithms*, Vol.1, No.1, 2000, pp.205-239.
- [10] Needleman, S. B. and Wunsch, C. D., A general method applicable to the search for similarities in the aminoacid sequence of two proteins, *Journal of Molecular Biology*, Vol. 48, 1970, pp.443-453.
- [11] Sellers, P. H., String Matching with Errors, *Journal of Algorithms*, Vol. 20, No. 1, 1980, pp.359-373.
- [12] Tarhio, J. and Ukkonen, E., Approximate Boyer-Moore String Matching, *SIAM Journal on Computing*, Vol. 22, No. 2, 1993, pp.243-260.
- [13] Wu, S. and Manber, U., Fast Text Searching: Allowing Errors, *Communications of the ACM*, Vol. 35, 1992, pp.83-91.